

C Programming - Assignment 12

Q.1 Create an Array of Employees and Search Employee by EmpID or Name

Problem Statement

Write a program to create an array of employee records. Each employee record should contain Employee ID, Name, and Salary. Search an employee either by Employee ID or by Name and display the employee details if found.

Concepts Used

- Structure
- Array of Structures
- Linear Search
- String Handling (`strcmp()`)
- Functions (Optional)
- Conditional Statements (`if`)
- Looping (`for`)

Step-by-Step Approach

1. Define a structure named `Employee`.
2. Create an array of employee structures.
3. Accept employee details from the user.
4. Display a menu to search by Employee ID or Name.
5. If searching by Employee ID, compare the entered ID with each employee record.
6. If searching by Name, compare the entered name using `strcmp()`.
7. If a match is found, display the employee details.
8. If no match is found, display **"Employee Not Found"**.

Test Cases

Test Case 1

Input

```
Employee Records:
101 Amit 50000
102 Neha 60000
103 Raj 55000
```

```
Choice: 1
Search EmpID: 102
```

Output

```
Employee Found  
ID : 102  
Name : Neha  
Salary : 60000.00
```

Test Case 2**Input**

```
Choice: 2  
Search Name: Raj
```

Output

```
Employee Found  
ID : 103  
Name : Raj  
Salary : 55000.00
```

Test Case 3**Input**

```
Choice: 1  
Search EmpID: 110
```

Output

```
Employee Not Found
```

Q.2 Modify Insertion Sort Algorithm to Sort the Array in Descending Order**Problem Statement**

Write a program to sort an array in **descending order** using the Insertion Sort algorithm.

Concepts Used

- Arrays
- Insertion Sort
- Looping (**for**, **while**)
- Conditional Statements (**while**)
- Swapping/Shifting Elements

Step-by-Step Approach

1. Accept the size of the array.
2. Accept array elements.
3. Consider the second element as the current element.
4. Compare it with previous elements.
5. Shift smaller elements one position to the right.
6. Insert the current element at the correct position.
7. Repeat until the entire array is sorted.
8. Display the sorted array in descending order.

Test Cases

Test Case 1

Input

```
5
10 30 20 50 40
```

Output

```
50 40 30 20 10
```

Test Case 2

Input

```
6
15 12 20 5 18 25
```

Output

```
25 20 18 15 12 5
```

Test Case 3

Input

```
4
8 6 4 2
```

Output

```
8 6 4 2
```

Q.3 Implement Binary Search Algorithm Using Recursion

Problem Statement

Write a recursive function to perform Binary Search on a sorted array and find the given element.

Concepts Used

- Arrays
- Binary Search
- Recursion
- Functions
- Conditional Statements (`if`)

Step-by-Step Approach

1. Accept the size of the sorted array.
2. Accept array elements.
3. Accept the element to search.
4. Call the recursive binary search function.
5. Find the middle element.
6. Compare the middle element with the search key.
7. If equal, return the position.
8. Otherwise, recursively search the left or right half.
9. Display the position if found; otherwise display **"Element Not Found"**.

Test Cases

Test Case 1

Input

10 20 30 40 50
Search: 30

Output

Element Found at Position 3

Test Case 2**Input**

5 10 15 20 25
Search: 25

Output

Element Found at Position 5

Test Case 3**Input**

2 4 6 8 10
Search: 7

Output

Element Not Found
